

```

import React, { useState } from "react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🏠 LOCAL TYPE ISOLATION GATEWAY
export type AvatarSize = "sm" | "md" | "lg" | "xl" | "2xl";
export type AvatarShape = "circle" | "square";

export interface AvatarProps extends React.HTMLAttributes<HTMLDivElement> {
  src?: string;
  alt?: string;
  name?: string;
  size?: AvatarSize;
  shape?: AvatarShape;
}

export function Avatar({
  src,
  alt,
  name = "System Operator",
  size = "md",
  shape = "circle",
  className,
  ...props
}: AvatarProps) {
  const [imageError, setImageError] = useState(false);
  const [isLoading, setIsLoaded] = useState(false);

  const initials = React.useMemo(() => {
    if (!name) return "";
    const parts = name.trim().split(/\s+/).filter(Boolean);
    if (parts.length === 1) {
      return parts[0].slice(0, 2).toUpperCase();
    }
    return (parts[0][0] + parts[parts.length - 1][0]).toUpperCase();
  }, [name]);

  const sizeClasses: Record<AvatarSize, string> = {
    sm: "w-8 h-8 text-xs",
    md: "w-10 h-10 text-sm",
    lg: "w-12 h-12 text-base",
    xl: "w-16 h-16 text-lg",
    "2xl": "w-20 h-20 text-xl",
  };
};

```

```

const shapeClasses: Record<AvatarShape, string> = {
  circle: "rounded-full",
  square: "rounded-[var(--radius-surface)]",
};

const containerClasses = cn(
  "relative inline-flex items-center justify-center overflow-hidden",
  "bg-muted/40",
  "border",
  "border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
  "hover:scale-105 transition-transform duration-200 ease-out",
  sizeClasses[size],
  shapeClasses[shape],
  className,
);

const imageClasses = cn(
  "absolute inset-0 w-full h-full object-cover",
  "transition-opacity duration-300 ease-out",
  isLoading && !imageError ? "opacity-100" : "opacity-0",
);

const fallbackClasses = cn(
  "relative z-10 flex items-center justify-center w-full h-full",
  "bg-muted/40",
  "text-muted-foreground",
  "font-medium",
  "transition-opacity duration-300 ease-out",
  !src || imageError ? "opacity-100" : "opacity-0",
);

const handleImageError = () => {
  setImageError(true);
};

const handleImageLoad = () => {
  setIsLoaded(true);
};

return (
  <div {...props} className={containerClasses}>
    {src && !imageError && (
      <img
        src={src}
        alt={alt || name}
        className={imageClasses}
        onError={handleImageError}
        onLoad={handleImageLoad}
      />
    )}
  </div>
);

```

```
    <div className={fallbackClasses} aria-hidden={!src && !imageError}>
      {initials}
    </div>
  </div>
);
}
```